

Operating Systems: Principles & Architecture

Chapter 1: Kernel Architectures & System Calls

The kernel is the foundational core of an operating system, maintaining complete control over hardware abstractions. Monolithic kernels execute all OS services—including process scheduling, IPC, memory management, and device drivers—in privileged supervisor ring 0.

Microkernels, conversely, strip the privileged core down to minimal primitives: primitive address space allocation, IPC, and thread dispatching. All device drivers and file systems operate in unprivileged user space.

Chapter 1.2: System Calls and Context Switching

When user software requires privileged hardware services, it invokes system calls (syscalls). This triggers a software interrupt, forcing the CPU from user mode (Ring 3) to kernel mode (Ring 0). Context switching saves program counter registers and TLB states.

Chapter 2: Process Scheduling Algorithms

Process scheduling manages the allocation of CPU execution time across competing threads. Common algorithms include First-Come-First-Served (FCFS), Shortest Job Next (SJN), and Round Robin (RR).

Algorithm	Preemptive	Time Complexity	Optimal For
Round Robin	Yes	$O(1)$	Time-sharing interactive systems
SJN	No	$O(N \log N)$	Minimizing average wait time
Completely Fair (CFS)	Yes	$O(\log N)$ red-black tree	Modern Linux scheduling

Chapter 2.2: Concurrency, Deadlocks & Banker Algorithm

A deadlock occurs when processes are blocked because each holds a resource while waiting for another. The four Coffman conditions necessary for deadlock are: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait.

Chapter 3: Memory Management & Virtual Memory

Virtual memory is an operating system mechanism that provides an idealized abstraction of the storage resources that are actually available on a given machine, creating a contiguous virtual address space mapped through page tables to non-contiguous physical DRAM frames.

Paging divides physical memory into fixed-size blocks called frames, and virtual address spaces into identical blocks called pages. When a process accesses a page that is not currently present in RAM, the Memory Management Unit (MMU) raises a page fault exception.

Chapter 3.2: Translation Lookaside Buffer (TLB)

Because walking multi-level page tables incurs significant memory latency, the CPU uses an associative cache called the Translation Lookaside Buffer (TLB). A TLB hit resolves virtual-to-physical address translation in a single clock cycle.

Chapter 3.3: Page Replacement & Beladys Anomaly

When physical frames are exhausted, the OS must evict pages using algorithms such as Least Recently Used (LRU), First-In-First-Out (FIFO), or the Clock algorithm. Beladys Anomaly demonstrates that increasing frame count in FIFO can counterintuitively increase total page faults.

Chapter 4: File Systems, Inodes & Journaling

An inode is an index node data structure describing a filesystem object such as a file or directory. It stores file metadata: permissions, owner, timestamps, and pointers to disk data blocks.

Chapter 4.2: Disk Scheduling & SSD Wear Leveling

Rotational disks utilize SCAN and C-SCAN elevator algorithms to minimize seek time. Solid State Drives (SSDs) use flash memory cells with flash translation layers (FTL) executing wear leveling to distribute erase cycles evenly.

Chapter 5: OS Security, Access Control & Protection Rings

Modern microprocessors provide hierarchical protection rings (Ring 0 to Ring 3). The OS enforces mandatory access control (MAC) and discretionary access control (DAC) to restrict privilege escalation and prevent unauthorized process interference.